👌 XAI Lecture 09

Adversarial Attacks on Explanations

FAMAF - UNC

First Semester 2026

eXplainable
Artificial Intelligence

🧉 Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/

🧉 Paper 1 ---

Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods

Dylan Slack*
University of California, Irvine
dslack@uci.edu

Sophie Hilgard*
Harvard University
ash798@g.harvard.edu

Emily Jia
Harvard University
ejia@college.harvard.edu

Sameer Singh
University of California, Irvine
sameer@uci.edu

Himabindu Lakkaraju
Harvard University
hlakkaraju@seas.harvard.edu

ABSTRACT

As machine learning black boxes are increasingly being deployed in domains such as healthcare and criminal justice, there is growing emphasis on building tools and techniques for explaining these black boxes in an interpretable manner. Such explanations are being leveraged by domain experts to diagnose systematic errors and underlying biases of black boxes. In this paper, we demonstrate that post hoc explanations techniques that rely on input perturbations, such as LIME and SHAP, are not reliable. Specifically, we propose a novel *scaffolding* technique that effectively hides the biases of

Explanation Methods. In *Proceedings of the 2020 AAAI/ACM Conference on AI, Ethics, and Society (AIES '20), February 7–8, 2020, New York, NY, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3375627.3375830

INTRODUCTION

Owing to the success of machine learning (ML) models, there has been an increasing interest in leveraging these models to aid decision makers (e.g., doctors, judges) in critical domains such as healthcare and criminal justice. The successful adoption of these models in domain-specific applications relies heavily on how well

(Slack et al. 2020)

🧉 Motivation ---

- ML is increasingly used for **critical** decisions
  - Healthcare, criminal justice, finance
- Decision makers must **understand model behavior** to:
  - Diagnose errors and potential biases
  - Decide when and how much to trust these models
- This motivates the need for **explainable AI (XAI)** tools
  - e.g., LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017)

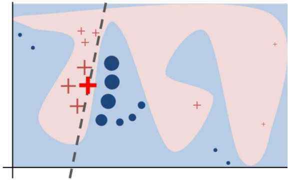
🧉 Motivation ---

- Trade-off between interpretability and accuracy
  - **Simple** models can be easily **interpreted** (e.g., linear regression)
  - **Complex** but also black-box model has much **better performance** (e.g., deep neural network)
- Can a ML method be both **interpretable** and **accurate**?
- *Post hoc* explanation can seemingly solve this problem:
  - First build **complex and accurate** ML models for good performance
  - Then use post hoc explanation for model interpretation
- The question is:
  - How robust and reliable is the *post hoc* explanation methods?

🧉 Contribution: A Framework to 'Fool' Post Hoc Explanations ---

- A **scaffolding** framework that hides the biases of any black box classifier
  - Targets **perturbation-based** explanation methods (LIME & SHAP)
  - Lets an adversary craft arbitrary, innocuous-looking explanations
- Evaluated on real-world datasets with extremely biased classifiers
  - COMPAS, Communities & Crime, German Credit
- **Key takeaway:** LIME and SHAP are **not robust enough** to detect discriminatory behavior in adversarial settings

🧉 Perturbation-based Post Hoc Explanation Method ---



Preliminaries & Background

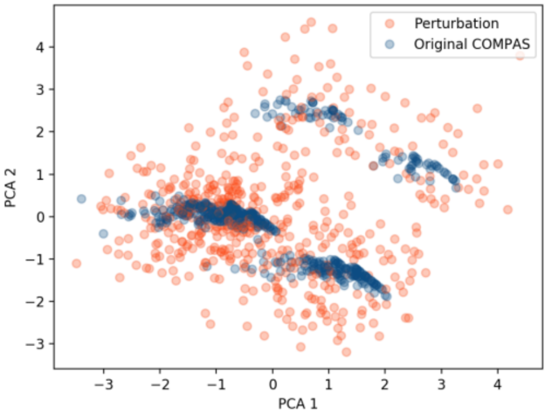
$$\arg\min_{g \in \mathcal{G}} L(f, g, \pi_x) + \Omega(g)$$

where the loss function L is defined as:

$$L(f, g, \pi_x) = \sum_{x' \in X'} [f(x') - g(x')]^2 \pi_x(x')$$

- f is the original classifier and x is the datapoint we want to explain
- g is the explanation we want to learn, $\Omega(g)$ is the "complexity" of g
- π is the proximity measure
- X' is a **synthetic dataset**, consisting of perturbations of x

🧉 Intuition ---



- LIME and SHAP explain predictions by **perturbing** input instances
- These perturbations often fall **outside the real data distribution** (out-of-distribution, OOD)
- Key observation: OOD points are **easily distinguishable** from real data

If we can detect whether a point is a perturbation or real data, we can make the model behave differently on each — and fool the explainer.

🧉 Approach: Set-up ---

Adversary would like to deploy a biased classifier f!

- **Background:** the biased model **f** uses sensitive attributes to make critical decisions
- **Requirement:** give access of black-box models to customers and regulators who use post-hoc explanations
- **Goal:** hide bias of the classifier **f**

🧉 Approach: Set-up ---

Goal: build an adversarial classifier $\tilde{e}$ that wraps any biased model f

- **Input:** dataset X sampled from the real-world distribution $\mathcal{X}_{dist}$
- **Output:** adversarial classifier $\tilde{e}$ defined as:

$$\tilde{e}(x) = \begin{cases} f(x) & \text{if } x \in \mathcal{X}_{dist} \quad (\text{real data} \rightarrow \text{biased behavior}) \\ \psi(x) & \text{otherwise} \quad (\text{perturbations} \rightarrow \text{innocuous behavior}) \end{cases}$$

where f is the **biased** classifier and ψ is an **unbiased** classifier

It's a Trap

On real data, $\tilde{e}$ behaves like f. On OOD perturbations, it looks fair.

🧉 Approach: OOD Detection ---

Key question: given input x, does it come from $\mathcal{X}_{dist}$ or is it a perturbation?

Solution: train a binary classifier `is_OOD(x)` using synthetic labels:

Data	Label
Original instances X	`False` (real data)
Perturbed instances X_p	`True` (OOD) — unless too close to X

- Perturbations are generated the same way LIME/SHAP would generate them
- Any off-the-shelf classifier works (authors use Random Forest)

Simple explained

`is_OOD(x)` is the gatekeeper: it decides whether $\tilde{e}$ runs f or ψ

How hard can it be to implement this? ---

```python
# Step 1: Generate perturbations (same method as LIME/SHAP)
X_perturbed = perturb(X)

# Step 2: Remove perturbations too close to real data
X_ood = [x for x in X_perturbed if not too_close(x, X)]

# Step 3: Build labeled dataset
X_train = X          + X_ood
y_train = [False]  + [True]     # False = real, True = OOD

# Step 4: Train OOD detector
is_OOD = RandomForest().fit(X_train, y_train)

# Step 5: Build adversarial classifier
def e(x):
    if not is_OOD(x):    # real data → biased behavior
```

Experiment: Set-up ---

90% training & 10% test

Dataset	Size	Features	Positive Class	Sensitive Feature
COMPAS	6172	criminal history, demographics, COMPAS risk score, jail and prison time	High Risk (81.4%)	African-American (51.4%)
Communities & Crime	1994	race, age, education, police demographics, marriage status, citizenship	Violent Crime Rate (50%)	White Population (continuous)
German Credit	1000	account	Good Customer	Male (69%)

🧉 Experiment: Results -- COMPAS ---

🧉 Experiment: Results -- Communities and Crime ---

🧉 Experiment: Results -- German Credit ---

🧉 Takeaway from Experiments ---

1. Accuracy of the OOD classifier $\rightarrow$ success of the adversarial attack
2. Adversarial classifiers to LIME are ineffective against SHAP explanations
   a. Any sufficiently accurate OOD classifier is sufficient to fool LIME, while fooling SHAP requires more accurate classifiers
3. SHAP less successful when using two features $\leftarrow$ local accuracy property
   a. Distribute attributions among several features

$$e(x) = \begin{cases} f(x), & \text{if } x \in \mathcal{X}_{dist} \\ \psi(x), & \text{otherwise} \end{cases}$$

🧉 Conclusions ---

- Main contribution: A framework for converting any black-box classifier into a *scaffolded* classifier that fools perturbation-based post-hoc explanation techniques like LIME and SHAP
- Effectiveness of this framework demonstrated on sensitive real-world data (criminal justice and credit scoring)
- Perturbation-based post-hoc explanation techniques are not sufficient to test whether classifiers discriminate based on sensitive attributes

🧉 Related Works ---

- Issues with post-hoc explanations:
  - [Doshi-Velez and Kim] identify explainability of predictions as a potentially useful feature of interpretable models.
  - [Lipton] and [Rudin] argues post-hoc explanations can be misleading and are not trustworthy for sensitive applications.
  - [Ghorbani et al.] and [Mittelstadt et al.] identified further weaknesses of post-hoc explanations.
- Adversarial explanations
  - [Dombrowski et al.] and [Heo et al.] show how to change saliency maps in arbitrary ways by imperceptibly changing inputs.

🧉 Q&A ---

- Are the experimental results sufficient to justify the conclusions?
  - In particular, how can we explain the discrepancy in results for LIME vs. SHAP?
- What about fooling other classes of post-hoc explanation methods?
  - Past work: gradient-based methods
- Alternatively, can one design post-hoc explanations that are *adversarially robust*?

🧉 Paper 2 ---

Explanations can be manipulated and geometry is to blame

Authors: Ann-Kathrin Dombrowski, Maximilian Alber, Christopher J. Anders, Marcel Ackermann, Klaus-Robert Müller, Pan Kessel

Presented by: Chelsea (Zixi) Chen, Tessa Han, Vignav Ramesh

(Dombrowski et al. 2019)

🧉 Introduction ---

🧉 Motivation ---

- Understand and verify aspects of ML models - Aid decision making in high-stakes scenarios

$\longrightarrow$ **Reliable** explanations of models!

Can we always trust model explanations?

🧉 Summary / Contribution ---

- Manipulate explanations! - Provide a theoretical understanding of such nonrobustness and derive a bound - Introduce smoothing to increase explanation robustness!

$$\|h(p)-h(p_0)\| \leq |\lambda_{max}|\, d_g(p,p_0) \leq \beta C\, d_g(p,p_0)$$

🧉 Background + Related Work ---

- Interpretation of Neural Networks is Fragile (Ghorbani, Abid, and Zou 2019)
  - Complex decision boundary
- The (un)reliability of saliency methods [Kindermans et al.]
  - Input invariance
- Sanity checks for saliency maps [Adebayo et al.]
  - Randomization test

- Fairwashing Explanations with Off-Manifold Detergent [Anders et al.]
  - Low-dimensional data manifold vs. High-dimensional embedding space

🧉 Methodology ---

🧉 Notation ---

- Neural network $g : \mathbb{R}^d \to \mathbb{R}^K$ with relu non-linearities
- Classifies input image x into K categories, predicted class $k = \arg\max_i g(x)_i$
- Explanation map: $h : \mathbb{R}^d \to \mathbb{R}^d$

- Target map: $h^t \in \mathbb{R}^d$
- Manipulated image: $x_{adv} = x + \delta x$

🧉 Properties of Manipulated Image ---

1. The output of the network stays approximately constant, i.e. $g(x_{adv}) \approx g(x)$.
2. The explanation is close to the target map, i.e. $h(x_{adv}) \approx h^t$.
3. The norm of the perturbation δx added to the input image is small, i.e. $\|\delta x\| = \|x_{adv} - x\| \ll 1$ and therefore not perceptible.

🧉 Explanation Methods ---

Propagation-based Gradient-based

- **Vanilla gradients:** $h(x) = \frac{\partial g}{\partial x}(x)$ - Quantifies how infinitesimal perturbations in each pixel change the prediction - **Gradient $\times$ Input:** $h(x) = x \odot \frac{\partial g}{\partial x}(x)$ - For linear models, this measure gives the exact contribution of each pixel to the prediction - **Integrated Gradients:** $h(x) = (x - \bar{x}) \odot \int_0^1 \frac{\partial g(\bar{x}+t(x-\bar{x}))}{\partial x} \mathrm{d}t$

- **Guided Backpropagation** - **Layer-wise Relevance Propagation** - **Pattern Attribution** - Standard backpropagation upon element-wise multiplication of the weights with learned patterns

🧉 Manipulation Method ---

Obtain manipulated images by optimizing the loss function:

$$\mathcal{L} = \|h(x_{adv}) - h^t\|^2 + \gamma\|g(x_{adv}) - g(x)\|^2$$

with respect to x_{adv} using gradient descent

- $h(x_{adv})$: manipulated explanation map
- h^t: target map
- γ: weighting hyperparameter
- $g(x_{adv})$: network output (manipulated input)
- $g(x)$: network output (original input)

🧉 Manipulation Method: Softplus Trick ---

- The gradient w.r.t. the input $\nabla h(x)$ of the explanation often depends on the vanishing second derivative of the relu non-linearities. This causes problems during optimization:

$$\partial_{x_{adv}} \| h(x_{adv}) - h^t \|^2 \propto \frac{\partial h}{\partial x_{adv}} = \frac{\partial^2 g}{\partial x_{adv}^2} \propto \mathrm{relu}'' = 0$$

- **Solution:** replace relu with softplus

$$\mathrm{softplus}_\beta(x) = \frac{1}{\beta} \log(1 + e^{\beta x})$$

🧉 Experiments ---

🧉 Experimental Setup ---

- Apply algorithm to 100 randomly selected images for each explanation method
- Use VGG-16 network pre-trained on ImageNet
- For each run, randomly select two images from the test set:
  - One of the two images is used to generate a target explanation map
  - The other image is perturbed by the algorithm with the goal of replicating the target using a few thousand iterations of gradient descent

- Comparable results obtained for ResNet-18, AlexNet, and Densenet-121 + CIFAR-10 dataset

Results: Visual Comparison ---

Results: Quantitative Metrics ---

🧉 Theoretical Analysis ---

🧉 Intuition ---

Why are explanations vulnerable and unreliable?

Large curvature of the NN output manifold!

🧉 Theoretical Bound ---

Theorem 1

Let $g : \mathbb{R}^d \to \mathbb{R}$ be a network with softplus$_\beta$ non-linearities and $\mathcal{U}_\epsilon(p) = \{x \in \mathbb{R}^d; \|x - p\| < \epsilon\}$ an environment of a point $p \in S$ such that $\mathcal{U}_\epsilon(p) \cap S$ is fully connected. Let g have bounded derivatives $\|\nabla g(x)\| \geq c$ for all $x \in \mathcal{U}_\epsilon(p) \cap S$. It then follows for all $p_0 \in \mathcal{U}_\epsilon(p) \cap S$ that

$$\|h(p) - h(p_0)\| \leq |\lambda_{max}|\, d_g(p, p_0) \leq \beta C\, d_g(p, p_0)$$

where λ_{max} is the principle curvature with the largest absolute value for any point in $\mathcal{U}_\epsilon(p) \cap S$ and the constant $C > 0$ depends on the weights of the neural network.

🧉 Robustness via Smoothing ---

$$\mathrm{softplus}_\beta(x) = \frac{1}{\beta}\log(1 + e^{\beta x})$$

Replacing relu with softplus reduces the curvature $\rightarrow$ more robust explanations

🧉 Smoothing: Connections to SmoothGrad ---

Theorem 2

For a one-layer neural network $g(x) = \text{relu}(w^T x)$ and its β-smoothed counterpart $g_\beta(x) = \text{softplus}_\beta(w^T x)$, it holds that

$$\mathbb{E}_{\epsilon \sim p_\beta}[\nabla g(x - \epsilon)] = \nabla g_{\frac{\beta}{\|w\|}}(x)$$

where $p_\beta(\epsilon) = \frac{\beta}{(e^{\beta\epsilon/2} + e^{-\beta\epsilon/2})^2}$.

- **SmoothGrad** $\equiv$ **β-smoothing**
- $\epsilon_i \approx \mathcal{N}(0, \sigma)$ with variance $\sigma = \log(2)\frac{\sqrt{2\pi}}{\beta}$

🧉 Robustness Experiments ---

🧉 Robustness Experiments ---

🧉 Conclusion ---

🧉 Critique ---

Strengths

- Thorough investigation: problem → reason → solution
- Extensive validation: various explanation methods, models, and datasets

Limitations

- Analyses focused on relu/softplus activation function
- Evaluation of robustness based on Pearson correlation coefficient

🧉 Future Directions & Food for Thought ---

Future directions

- Extend empirical analyses to other tasks and data modalities
- Generalize theoretical analyses to propagation-based methods
- Modify model training process to make NNs less vulnerable to explanation manipulation
  - Low-curvature models [Srinivas et al., NeurIPS 2022]

Food for thought

- Might there be other reasons for explanations being sensitive to manipulation?
- What are other ways to evaluate robustness of explanations?
- Is it a good idea to trade-off faithfulness for better robustness?

Thank You!

References --- |

Dombrowski, Ann-Kathrin, Maximilian Alber, Christopher J. Anders, Marcel Ackermann, Klaus-Robert Müller, and Pan Kessel. 2019. "Explanations Can Be Manipulated and Geometry Is to Blame." In *Advances in Neural Information Processing Systems*.

Ghorbani, Amirata, Abubakar Abid, and James Zou. 2019. "Interpretation of Neural Networks Is Fragile." In *Proceedings of the Aaai Conference on Artificial Intelligence*.

Slack, Dylan, Sophie Hilgard, Emily Jia, Sameer Singh, and Himabindu Lakkaraju. 2020. "Fooling LIME and SHAP: Adversarial Attacks on Post Hoc Explanation Methods." In *Proceedings of the Aaai/Acm Conference on Ai, Ethics, and Society*.